

RACTF 2020 Crypto - Mysterious Masquerading Message

deuterium — 2020-06-09

Canonical HTML: [https://deuterium.github.io/WriteUps/2020/ractf/crypto/Mysterious`Masquerading`Message/2020-06-09-RACTF-2020-Mysterious-masquerading-message.html](https://deuterium.github.io/WriteUps/2020/ractf/crypto/Mysterious%20Masquerading%20Message/2020-06-09-RACTF-2020-Mysterious-masquerading-message.html)

Abstract

CTF writeup: RACTF 2020's Mysterious Masquerading Message wears an SSH private-key costume. Decode its clues to find the key and IV for AES-CBC decryption.

Mysteriuos Masquerading Message

We found a file that looks to be like an ssh private key... but it doesn't seem quite right. Maybe you can shed some light on it?

The contents of `id`rsa.txt` read

```
tbbq yhpx:)
```

```
-----BEGIN OPENSSH PRIVATE KEY-----
SWYgeW91IGFyZSB5ZWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZ
pZ3VyZWQgb3V0IHRoYXQgaXQgd2Fzbid0IGFjdHVhbGx5IGFuIFNTSCBrZX
kgYnV0IGFyZGZlZ3Vpc2UuIFNvIHRvdSB0YXZlIG1hZGUgaXQgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgZGhhZCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoXmZkaW5kIHRoaW5raW5nIGFib3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHRvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNoZGVzciBjbHV
lcYwgYnV0IHRvdSBjYW50IGZpbmQgYW55LiBZb3UgYXZlIHNoYXJ0aW5nIH
RvIGdlbCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaW5nIGhhdmVudG9kbWVudG9kbWVudG9kbWVudG9kbWV
IGRvIHdpdGggdGhlIGNoYXJ0aW5nZSwgZXhjaXB0ICJ3ZSwgIGRvbmUgeW9
1IGhhdmUgZ290IHRoaXMGZmFyIi4gW91IHNoYXJ0IHNoYXJ0aW5nIGRlc2
tzLzCBhbmQgc29vbiB0aGUgbW9uaXRvciB3aW5nIGZvbGxvdy4gW91IGFyZ
SB3b25kZXJpbmcgd2hlcmUgdGhpcyBpcyBnb2luZyBhbmQgcVhbGZaW5n
IGl0J3MgY29taW5nIHRvIHRoZSB1bmQgb2YgdGhlIHhbmFncmFwaCwgYW5
kIHRvdSBtaWdodCBub3QgaGF2ZSBzZWVuIGFueXRoZW5nLiBBIgghdmUgZ2
l2ZW4geW91IHNoYXJ0aW5nIGFueXRoZW5nIGFueXRoZW5nIGFueXRoZW5n
HNvbWV0aGluZyBlbnNlIGFzIHdlbGwgZ29vZCBsdWNoLiAKNjk2ZTY1NjU2
NDc0NmY2ZjcwNjU2ZTZjNmY2MzZiNzMKNjk2ZTY5NzQ2OTYxNmM2OTczNjE
3NDY5NmY2ZTMxMzI=
-----END OPENSSH PRIVATE KEY-----
```

```
00111001 00110000 00111001 00111000 00111000 01100011 00111001 01100010
01100101 01100110 01100101 00110101 01100101 01100001 00110011 01100110
00110101 01100001 00111001 00110001 01100101 01100110 01100110 01100101
00110000 00110011 00110000 00110110 00110000 01100001 00111000 00110111
```

00110001	00110100	01100100	01100110	01100011	00110010	00110000	00110000
00111000	00111000	00110100	00110001	00110101	00110101	00110111	00110000
01100010	00110011	00111001	00110100	01100011	01100101	00111001	01100011
01100100	00110011	00110010	01100010	01100101	00110111	00110001	00111000

Now the contents of proclaimed private key seems weird, I tried giving it a base64 decode

```
import base64
priv_key = """
SWYgeW91IGFyZSByZWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZ
pZ3VyZWQgb3V0IHRoYXQgaXQgd2Fzbid0IGFjdHVhbGx5IGFuIFNTSCBrZX
kgYnV0IGEGZGlzZ3Vpc2UuIFNvIHlvdSB0YXZlIG1hZGUgaXQgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGRvIHdpdGggdGhlIGNoYXxsZW5nZSwgZWhjaXZlIHRvIGRvbmUgeW91IGhhd
mUgZ290IHRoaXMsIHRvIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcyBmY
XIgYW5kIGZvciB0aGF0IEkgc2F5IHdlbGwgZG9uZS4gSXQgd2Fzbid0IHZl
cnkgaGFyZCwgdGhhdBmVudGhhdCBJIGtub3csIGJldCBuZXZlcnRoZWxlcnMgeW91IGh
hdmUgc3RpbGwgbWZkaW5nIHRoaXMsIHRoZW4geW91IHByb2JhYmx5IGZpZ3VyZWQgb3V0I
GFubm95aW5nIHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpcywgYW5kIHlvdSB3
YW50IHRvIHJlYWQgdGhlIHdob2x1IHRoaW5nIHRvIGNoZWNrIGZvciBjbHV
lcYwgYnV0IHlvdSBjYW50IGZpbmQgYW55LiBZb3UgYXJlIHNoYXJ0aW5nIH
RvIGdldCBmcnVzdHJhdGVkIGF0IHRoZSBwZXJzb24gd2hvIG1hZGUgdGhpc
yBhcyB0aGV5IHNoaWxsIGhhdmVudGhhdGVudGlvbmVkaGFueXRoZW4geW91IHRv
IGR
```

Voila! It reads

If you are reading this, then you probably figured out that it wasn't actually an SSH key but a disguise.

So you have made it this far and for that I say well done. It wasn't very hard, that I know, but nevertheless

you have still made it here so congrats. Now you are probably reading this and thinking about annoying the

person who made this, and you want to read the whole thing to check for clues, but you cant find any. You

are starting to get frustrated at the person who made this as they still haven't mentioned anything to do

with the challenge, except "well done you have got this far". You start slamming desks, and soon the monitor

will follow. You are wondering where this is going and realising it's coming to the end of the paragraph,

and you might not have seen anything. I have given you some things, although you will need something else

as well good luck.

696e656564746f6f70656e6c6f636b73

696e697469616c69736174696f6e3132

trying out hex decoding the last two strings

```
part_1 = "696e656564746f6f70656e6c6f636b73"
part_2 = "696e697469616c69736174696f6e3132"

p1 = bytes.fromhex(part_1)
p2 = bytes.fromhex(part_2)
p3 = bytes.fromhex(bin_decoded_m2)
print(p1.decode())
print(p2.decode())
```

```
ineedtoopenlocks
initialisation12
```

hmm, no hint whats going on.

Lets try decoding the binary data at the end of the private key

```
m2 = ""
00111001 00110000 00111001 00111000 00111000 01100011 00111001 01100010
01100101 01100110 01100101 00110101 01100101 01100001 00110011 01100110
00110101 01100001 00111001 00110001 01100101 01100110 01100110 01100101
00110000 00110011 00110000 00110110 00110000 01100001 00111000 00110111
00110001 00110100 01100100 01100110 01100011 00110010 00110000 00110000
00111000 00111000 00110100 00110001 00110101 00110101 00110111 00110000
01100010 00110011 00111001 00110100 01100011 01100101 00111001 01100011
01100100 00110011 00110010 01100010 01100101 00110111 00110001 00111000
""

bin_decoded_m2 = "".join( chr(int(i,2)) for i in m2.split())
print(bin_decoded_m2)
```

Producing the hex string

```
90988c9befe5ea3f5a91effe03060a8714dfc20088415570b394ce9cd32be718
```

Now this got confusing at this point of time. I got 3 strings and not knowing what to do with them. I failed so many times XORing the strings creatively LOL.

Later did I realise the ineedtoopenlocks is a hint to **KEY**

And initialisation12 is a huge hint to **IV**

And the third string could be our ciphertext. By this time one would consider it to be AES encrypted.

Which mode?

The mode which has IV and should be easy to guess :- CBC

```
from Crypto.Cipher import AES
part_1 = "696e656564746f6f70656e6c6f636b73"
part_2 = "696e697469616c69736174696f6e3132"
bin_decoded_m2 = "90988c9befe5ea3f5a91effe03060a8714dfc20088415570b394ce9cd32be718"
p1 = bytes.fromhex(part_1)
p2 = bytes.fromhex(part_2)
```

```
p3 = bytes.fromhex(bin_decoded_m2)

cipher = AES.new(p1, AES.MODE_CBC, IV=p2)
flag = cipher.decrypt(p3)
print(flag.decode())
```

ractf{3Asy_FlaG_0n_aEs_rAcTf} and we get our flag sans padding!

| Interactive controls are omitted from this PDF. See the canonical HTML version.